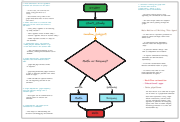
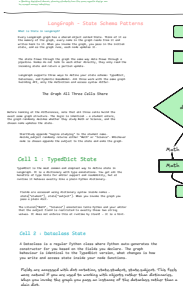




LangGraph — State Schema Patterns

What is State in LangGraph?

Every LangGraph graph has a shared object called State. Think of it as a whiteboard that every node can read from and write to. When a node finishes it returns a dict. What you choose for that dict, and how the framework merges it into the existing state, is entirely up to you.

The state flows through the graph the same way data flows through a function: nodes do not call one another directly. They only read the incoming state and return a partial update.

LangGraph supports three ways to define your state schema: TypedDict, Dataclass, and Pydantic BaseModel. All three work with the same graph building API. Only the behaviour and access syntax differs.

The Graph All Three Cells Share

Before looking at the differences, note that all three cells build the exact same graph structure. The topology is identical — a START edge, two analysis nodes, and an END edge. Only the way each is defined, and the shape data updates, differ.

Conditional routing (route_decision) is the shared idea: a function reads the state and returns a string that names the next node. Whatever that is always appends the output to the state, and adds the graph

Cell 1 : TypedDict State

TypedDict is the most common and simplest way to define state in LangGraph. It is a dictionary with type annotations. You get the benefit of knowing what keys exist and what types they hold, but no runtime or instance enforcement. Data is plain Python dictionary.

Fields are accessed using dictionary syntax: state['key'].
Nodes return partial dicts: {'key': 'value'}. They are merged into the graph state automatically.

The returned dict, 'revised' overwrites only that key. Every other field in the state remains exactly as it was before the node ran.

Cell 2 : Dataclass State

A Dataclass is a regular Python class where Python auto-generates the constructor for you based on the fields you declare. The graph behaviour is identical to TypedDict. The only difference is how you write and access state inside your node functions.

Fields are accessed with dot notation, state.student_name, not state['student_name']. This feels more natural if you are used to working with objects rather than dictionaries. Nodes still return plain dicts. The graph then merges an instance of this dataclass rather than a plain dict.

The key thing to understand is that even though the state is a dataclass object, nodes still return a plain dictionary as their update. LangGraph merges that dict back into the state for you.

Cell 3 : Pydantic BaseModel State

Pydantic brings runtime validation to your state. When a node returns an update, Pydantic checks that the values match the declared types and will attempt to coerce them if they do not match exactly. For example, if a node returns an integer where a string is expected, Pydantic will convert it automatically.

This makes Pydantic the right choice when your graph receives inputs from external sources, APIs, user forms, or other systems, where the data may not be perfectly typed. It catches bad data at the boundary, rather than letting it silently corrupt your graph's behaviour.

Fields are accessed with dot notation just like a dataclass, state.student_name, not state['student_name']. Incorrect usage either a plain dict or a model instance.

Comparing the Three Schemas

The graph structure, the node logic, and the conditional routing are identical across all three cells. What changes is narrow and specific.

With TypedDict, you are working with dictionaries throughout. State access uses bracket notation. Invocation takes a plain dict. There is no validation.

With Dataclass, you are working with objects. State access uses dot notation. Invocation takes a single instance. There is no validation.

With Pydantic, you are working with objects that validate themselves. State access uses dot notation. Invocation accepts either a dict or an instance. Pydantic validates and coerces types at runtime.

Choose TypedDict when you want something quick and lightweight. Choose Dataclass when you prefer attribute access and cleaner-looking node functions. Choose Pydantic when your graph deals with external data and you need runtime guarantees about what is in your state.

Chains with LangGraph

What is a Chain?

A chain in LangGraph is a sequence of nodes connected by edges. Data flows from START through one or more nodes and reaches END. Each node is a Python function that receives the current state, does some work, and returns an update. The graph manages the flow; you just define the pieces.

Three building blocks make up every chain: nodes, normal edges, and conditional edges. A normal edge always goes from A to B. A conditional edge lets the graph decide at runtime which node to go to next based on the state.

Four Concepts This Code Introduces

The notebook builds one idea on top of another. Understanding why each piece exists makes the code obvious.

1. Chat Messages as State

The graph needs to remember the conversation. LangGraph uses LangChain's message types to represent each turn. A HumanMessage carries what the user said. An AIMessage carries what the model replied. These messages are stored in the state and passed in to every node so the model always has the full conversation history available.

In the book example, the conversation starts with a welcome message from the bot and a request from the Customer. That entire list is the state.

2. Chat Models in Graph Nodes

The LLM is not called outside the graph. It lives inside a node. The node receives the current state, passes the list of messages to the LLM, and returns the response as an update. This means the call to the LLM call within the graph's control flow, meaning you can add routing, retries, or tool calls around it cleanly.

3. Binding Tools to the LLM

An LLM on its own can only generate text. Tools let it interact with the outside world, such as calling an API, querying a database, or running a calculation. You bind tools to the LLM using bind_tools(). From that point on, when the model receives a query that requires a tool, it does not answer directly; it outputs a tool call request instead, specifying which tool to use and with what arguments.

In the book example, calculate_interest is bound to the LLM. When a user asks about interest on a balance, the model outputs a tool call rather than guessing the answer.

4. Executing Tool Calls in Graph Nodes

The model requesting a tool and the tool actually running are two separate steps. LangGraph handles the second step with ToolNode automatically. You pass your list of tools to ToolNode and add it as a node in the graph. tools_condition is a built-in routing function that checks whether the last AI message contains a tool call. If yes, it routes to the tools node; if no, it routes to END.

This separation is intentional. It means the graph can inspect, log, or modify a tool call before it runs.

How the Graph Knows Across the Notebook

The first graph is simple: START goes to the LLM node, which goes to END. It proves the LLM works inside a graph and can handle the conversation state. There is no tool execution here; if the model produces a tool call, it just sits in the final message unexecuted.

The node builds the graph in five stages, and it helps to see this as deliberate progression.

The second graph adds the tools node and the conditional edge. Now when the model produces a tool call, the graph notices it, runs the tool, and passes the result. When the model gives a direct response, the graph ends immediately. The same LLM node handles both cases; only the routing changes.

The Reducer: Why add_messages Matters

State in LangGraph is updated by merging. When a node returns a dict, LangGraph merges it into the existing state. For most fields that means the new value replaces the old one. For a list of messages that would be a problem; you would lose the conversation history every time a node ran.

add_messages is a reducer that changes this behaviour. Annotating the messages field with it tells LangGraph to append new messages rather than replace the whole list. This is what keeps the conversation intact as it moves through nodes.

Agent Basics

Agents are programs like chatbots. They can make decisions from user input in natural language. They do not need human help. They can either call a tool or give a final answer.

Conditional Edges and Tool Nodes

These are the rules and points that guide the agent. They decide when the agent should use a tool and when it should stop and give the answer.

ReACT Architecture

This design has three clear steps:

- Observe: The agent looks at the tool's output. It does not end the chat yet.
- Act: The agent takes action. It calls a tool when needed.
- Reason: The agent thinks about the output. It decides the next step.

This loop can repeat until the agent is ready to give the final answer.

LangGraph ReACT Loop Notes

ReACT stands for Reason + Act + Observe + Answer. It is a structured way for an agent to handle queries by combining reasoning with tool use.

The loop ensures that the agent does not just guess answers but can call external tools, observe their outputs, and then respond with accurate information.

- HumanMessage = user input
- Agent Node = reasoning step
- Tool Node = action step
- ToolMessage = observation step
- AIMessage = final answer

What is the ReACT Loop?

Step by Step in the BankBot Example

1. HumanMessage

The user input is wrapped as a HumanMessage. For example: "weather in pune". This tells the agent that the message came from the user.

2. Agent Node (Reason)

The LLM reads the HumanMessage and reasons about what to do. In this case, it realizes that weather data must come from a search tool. It generates a structured tool call request.

3. Tool Node (Act)

The graph routes to the ToolNode because the agent requested a tool. The tool executes the query, such as calling a search API for "weather in pune".

4. ToolMessage (Observe)

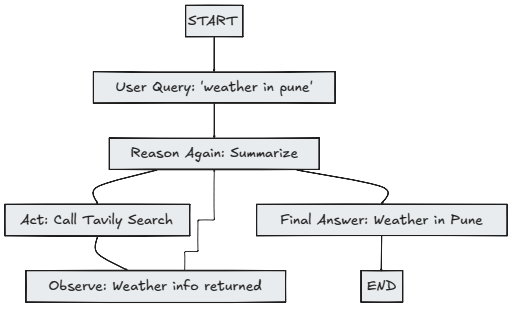
The tool's output is wrapped as a ToolMessage. This tells the agent that the information came from a tool. For example: "Current temperature in Pune is 34°C, partly cloudy."

5. Agent Node Again (Reason Again)

The LLM sees the ToolMessage and reasons again. Now it has the data, so it decides to summarize it clearly for the user. If more data was needed, it could loop back to call another tool.

6. AIMessage (Final Answer)

The agent produces the final structured reply as an AIMessage. For example: "The weather in Pune is 34°C with partly cloudy skies." This is what the user sees as the answer.

This loop shows how LangGraph encodes the ReACT pattern:

Reason → Act → Observe → Reason Again → Answer.
It makes clear that the agent is not guessing but following a disciplined process.